

Overall Merit: Accept

Summary This is a tutorial that puts a single vocabulary on the relaxed memory models that had piled up by 1995. After defining sequential consistency, the authors use Figure 5 to show how three normal hardware optimizations (write buffers with bypassing, overlapping writes, non-blocking reads) each break SC for Dekker-style code.

The bulk of the paper is a taxonomy. Each model is described by which program orders it relaxes ($W \rightarrow R$, $W \rightarrow W$, $R \rightarrow RW$) and which write-atomicity rules it relaxes. Figure 8 lays out IBM 370, TSO, PC, PSO, WO, RCsc, RCpc, Alpha, RMO, and PowerPC on the same grid with their safety nets. Section 7 then argues for a programmer-centric view: the programmer labels operations involved in races as synchronization, the system runs whatever optimization SC permits, and race-free code sees SC.

Strengths

1. The unified terminology actually works. Putting Alpha, RMO, and PowerPC on the same axes (Figure 8) makes it obvious that PowerPC is the most permissive and Alpha sits one column over because it preserves its own write order. Comparing models used to mean translating between three notations.
2. The myth table in Figure 2 is unusually direct for a tutorial. It names specific misconceptions (“the model only affects hardware,” “relaxed models require extra synchronization”) and points to where the paper refutes each. That saves the reader from walking into Section 6 with the wrong mental model.

Weaknesses

1. The paper claims relaxed models give better performance than SC but cites this to other work without reproducing any numbers. For 20 pages on the cost of SC, even one speedup chart would have made the motivation concrete.
2. The compiler treatment is thin. Section 5.3 says the compiler must preserve program order; Section 6.4.4 says relaxed models give the compiler back its uniprocessor optimizations between fences. What that actually means in C (what is a fence? what about register allocation across a synchronization?) is left to the references.

Detailed Comments Reading this in 2026, Section 7 is the part that aged best. The system-centric models in Section 6 are mostly historical now (TSO survives because x86 implements it, PSO

and IBM 370 are gone), while data-race-free is the foundation of every modern language memory model. Figure 6 is also the cleanest example I have seen of why serializing writes to the same location is not enough on its own; it forced me to separate “writes to one location are seen in the same order” from “a write looks atomic,” which informal descriptions tend to conflate.

Discussion Questions

1. DRF0 only guarantees SC for race-free programs. But useful concurrent code (lock-free queues, seqlocks) deliberately races on synchronization variables. Is that why C++11 added explicit memory orders (**relaxed**, **acquire**, **release**) on top of the DRF contract?
2. Figure 8 places models on a grid, but the rows are not totally ordered. What does it mean for one model to be “stronger” than another when they relax incomparable orderings?
3. The paper assumes the boundary of interest is between programmer and hardware. With JIT compilers and managed runtimes, the model now has to be respected at multiple layers. How would you extend the framework to cover an x86 CPU running JVM bytecode written against the Java memory model?